

torch.empty_strided#

https://pytorch.org/docs/stable/generated/torch.empty_strided.html

Description:

Creates a tensor with the specified size and stride and filled with undefined data. If the constructed tensor is “overlapped” (with multiple indices referring to the same element in memory) its behavior is undefined. If `torch.use_deterministic_algorithms()` and `torch.utils.deterministic.fill_uninitialized_memory` are both set to `True`, the output tensor is initialized to prevent any possible nondeterministic behavior from using the data as an input to an operation. Floating point and complex tensors are filled with NaN, and integer tensors are filled with the maximum value.

Function Signature

```
torch.empty_strided(size, stride, *, dtype=None,
layout=None, device=None, requires_grad=False,
pin_memory=False) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

`dtype` (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see `torch.set_default_dtype()`). `layout` (torch.layout, optional) – the desired layout of returned Tensor. Default: `torch.strided`. `device` (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see `torch.set_default_device()`). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types. `requires_grad` (bool, optional) – If autograd should record operations on the returned tensor. Default: False. `pin_memory` (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.

Code Examples

```
>>> a = torch.empty_strided((2, 3), (1, 2))
>>> a
tensor([[8.9683e-44, 4.4842e-44, 5.1239e+07],
        [0.0000e+00, 0.0000e+00, 3.0705e-41]])
>>> a.stride()
(1, 2)
>>> a.size()
torch.Size([2, 3])
```

Notes & Warnings

WARNING

If the constructed tensor is “overlapped” (with multiple indices referring to the same element in memory) its behavior is undefined.

NOTE

If `torch.use_deterministic_algorithms()` and `torch.utils.deterministic.fill_uninitialized_memory` are both set to `True`, the output tensor is initialized to prevent any possible nondeterministic behavior from using the data as an input to an operation. Floating point and complex tensors are filled with NaN, and integer tensors are filled with the maximum value.

Detailed Documentation

Documentation

Creates a tensor with the specified size and stride and filled with undefined data.

If the constructed tensor is “overlapped” (with multiple indices referring to the same element in memory) its behavior is undefined.

If `torch.use_deterministic_algorithms()` and `torch.utils.deterministic.fill_uninitialized_memory` are both set to `True`, the output tensor is initialized to prevent any possible nondeterministic behavior from using the data as an input to an operation. Floating point and complex tensors are filled with NaN, and

integer tensors are filled with the maximum value.

size (tuple of int) – the shape of the output tensor

stride (tuple of int) – the strides of the output tensor

dtype (torch.dtype, optional) – the desired data type of returned tensor. Default: if None, uses a global default (see torch.set_default_dtype()).

layout (torch.layout, optional) – the desired layout of returned Tensor. Default: torch.strided.

device (torch.device, optional) – the desired device of returned tensor. Default: if None, uses the current device for the default tensor type (see torch.set_default_device()). device will be the CPU for CPU tensor types and the current CUDA device for CUDA tensor types.

requires_grad (bool, optional) – If autograd should record operations on the returned tensor. Default: False.

pin_memory (bool, optional) – If set, returned tensor would be allocated in the pinned memory. Works only for CPU tensors. Default: False.